Relational Algebra in DBMS

1. What is Relational Algebra?

Relational Algebra is a formal query language used to access and manipulate data in a relational database. It was introduced by **E. F. Codd in 1970** as the theoretical foundation for the relational database model.

> ⭐ **EXAM DEFINITION**
> **Relational Algebra** is a procedural query language consisting of mathematical operations used to retrieve and manipulate data from relational databases.

2. Why is Relational Algebra Called Procedural?

It is called **procedural** because the user specifies both *what* data is needed and *how* (the sequence of operations) to obtain it.

- **WHAT:** Specifies the final required result.
- **HOW:** Specifies the step-by-step procedural operations/expressions to achieve it.

Contrast: SQL is largely declarative (you specify *what* you want, leaving the query optimizer to decide *how* to fetch it).

3. Why Relational Algebra Matters Today

Although SQL is used in practice, Relational Algebra provides the core mathematical foundation for database theory, query optimization, and join execution.

Theoretical-to-Practical Progression:
Relational Model → Relational Algebra → Query Optimization → SQL / RDBMS Engines

4. Relational Algebra vs. SQL

Feature	Relational Algebra	SQL
Nature	Formal / Mathematical / Procedural	Practical / Commercial / Declarative
Syntax	Mathematical symbols (σ, π, $\times$, $\cup$, $-$, ρ, $\bowtie$)	Keywords (`SELECT`, `FROM`, `WHERE`, `JOIN`)
Row Selection	Selection (σ)	`WHERE` clause
Column Projection	Projection (π)	`SELECT column_list`

5. Fundamental Operations (The Core 6)

There are **6 basic/fundamental operations** from which all other operations can be derived.

6.1 Selection (σ)

Selects **rows (tuples)** satisfying a given predicate/condition.

Example: $\sigma_{Dept='IT'}(Employee)$

```
SELECT * FROM Employee
WHERE Dept = 'IT';
```

6.4 Union ($\cup$)

Combines tuples from two relations. Requires **Union Compatibility** (same degree & compatible domains).

Formula: $R \cup S$ (Duplicates automatically removed).

> 🧠 **Selection → Selects Rows**

6.2 Projection (π)

Selects specific **columns (attributes)** and eliminates duplicates.

Example: $\pi_{Emp\_Name,\ Address}(Employee)$

```
SELECT DISTINCT Emp_Name, Address
FROM Employee;
```

> 🧠 **Projection → Picks Columns**

6.3 Cartesian Product (×)

Combines every row of relation R with every row of relation S.

If $|R| = m$ and $|S| = n$, then: $|R \times S| = m \times n$

> 🧠 **Cartesian Product → Every Combination**

6.5 Set Difference (-)

Returns tuples present in relation R but **not** in S. Requires union compatibility.

Non-commutative: $R - S \neq S - R$

6.6 Rename (ρ)

Renames a relation or its attributes. Vital for self-joins.

Syntax: $\rho_{X(A1,\ A2)}(R)$ renames relation R to X with attributes $A1, A2$.

> 🧠 **Rename → Give New Name**

7. Derived Operations

These operations can be expressed using combinations of fundamental operations but are defined separately for convenience and query optimization.

7.1 Join (⋈)

Combines related tuples from two relations based on a matching condition.

- **Theta / Equi Join:** Cartesian product followed by selection: $\sigma_c(R \times S)$.
- **Natural Join (⋈):** Automatically joins on all common attribute names and removes duplicate common columns.
- **Outer Joins (Left ⟕, Right ⟖, Full &fulljoin;):** Preserves unmatched rows by padding with NULL.

7.2 Intersection (∩)

Returns tuples common to both relations.

Derivation: $R \cap S = R - (R - S)$

7.3 Division (÷)

Used for queries containing universal quantification ("FOR ALL").

Example: "Find students who completed *all* required courses."

8. Fundamental vs. Derived Operations Summary

Fundamental Operation	Symbol	Derived / Common Operation	Symbol
Selection	σ	Join	⋈
Projection	π	Intersection	∩
Cartesian Product	×	Division	÷

Fundamental Operation	Symbol	Derived / Common Operation	Symbol
Union	∪	Outer Joins	$\&leftouterjoin;$, $\&rightouterjoin;$, $\&fullouterjoin;$
Set Difference	-	-	-
Rename	ρ	-	-

🧠 **GATE MNEMONIC TRICK**

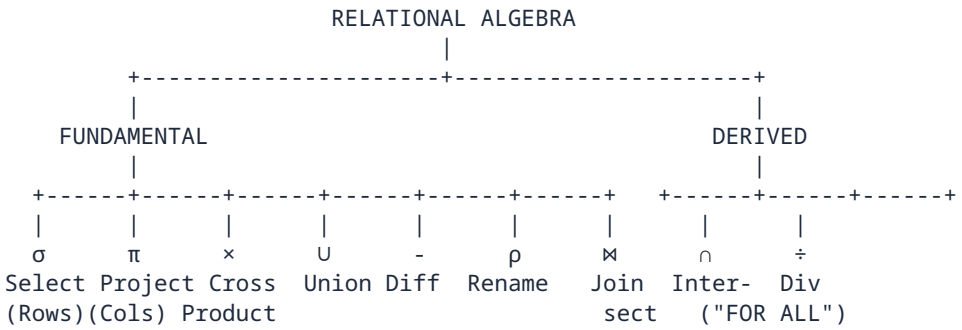
The 6 Fundamental Operations = S P C U D R

Selection (σ) • **P**rojection (π) • **C**artesian Product (×) • **U**nion (∪) • **D**ifference (-) • **R**ename (ρ)

9. Selection vs. Projection (High-Frequency Exam Topic)

Selection (σ)	Projection (π)
Filters **Rows (Tuples)**	Filters **Columns (Attributes)**
Uses a conditional predicate (e.g., $Age > 20$)	Specifies explicit attribute list
Output cardinality ≤ input cardinality	Degree = number of projected attributes
Equivalent to SQL `WHERE`	Equivalent to SQL `SELECT column_list`

10. Structural Map & Classification



⚠️ Exam Traps & Pitfalls

WATCH OUT FOR THESE COMMON TEST TRAPS:

- **Trap 1: Procedural Nature:** Relational Algebra specifies the sequence of operations (HOW), whereas SQL specifies WHAT.
- **Trap 2: Selection vs Projection Symbol:** σ = Selection (Rows), π = Projection (Columns). Do not swap them!
- **Trap 3: Set Difference Order:** $R - S \neq S - R$. Set difference is directional and non-commutative.
- **Trap 4: Union Compatibility:** ∪ , ∩, and - require both relations to have the same number of attributes with matching domains.
- **Trap 5: Cartesian Product Cardinality:** If $|R| = m$ and $|S| = n$, $|R \times S| = m \times n$. Degree of $R \times S$ is $Degree(R) + Degree(S)$.

⭐ Complete Relational Algebra Cheat Sheet

Operation	Symbol	Primary Purpose	Key Memory Cue
Selection	σ	Filter rows matching condition	Rows / WHERE
Projection	π	Extract specified columns	Columns / SELECT
Cartesian Product	×	Combine every tuple pair	Multiply ($m \times n$)
Union	∪	Combine tuples from both sets	OR / Merge
Set Difference	-	Tuples in first but not second	Remove / Exceptional
Rename	ρ	Rename relation/attributes	Alias / Self-Join
Join	⋈	Combine related tuples on condition	Match / Cross + Select
Intersection	∩	Common tuples in both	AND / Overlap
Division	÷	Find tuples satisfying "FOR ALL"	Universal Quantification

🚀 **ONE-MINUTE REVISION SUMMARY**

Relational Algebra = Procedural + Mathematical + Formal Query Language.

6 Fundamental Operations: σ, π, ×, ∪, -, ρ | **Derived Operations:** ⋈, ∩, ÷

Shortcuts: σ = Rows • π = Columns • × = All Combinations • ⋈ = Matching Data • ÷ = "FOR ALL"

⭐ **Golden Rule: Master Relational Algebra first; SQL logic will become intuitive!**